

Sistema de Registro Escolar ClassFlow

Presentación del sistema desarrollado con
arquitectura de microservicios

Presentado por:

Angelo Millan & Evens Reneus



Resumen

ClassFlow es un sistema de gestión educativa diseñado para centralizar y automatizar procesos académicos: usuarios, cursos, evaluaciones, asistencia, mensajería y notificaciones en una sola plataforma integrada.

Microservicios con Java Spring Boot y React 18 +
TypeScript

Orquestación con Docker Compose y despliegue en
Kubernetes

Seguridad JWT con roles: ADMIN, DOCENTE,
ESTUDIANTE, APODERADO

Arquitectura basada en microservicios independientes, cada uno con su propia base de datos PostgreSQL, comunicación REST vía API Gateway y frontend SPA desacoplado del backend.

Arquitectura del Sistema Vista General

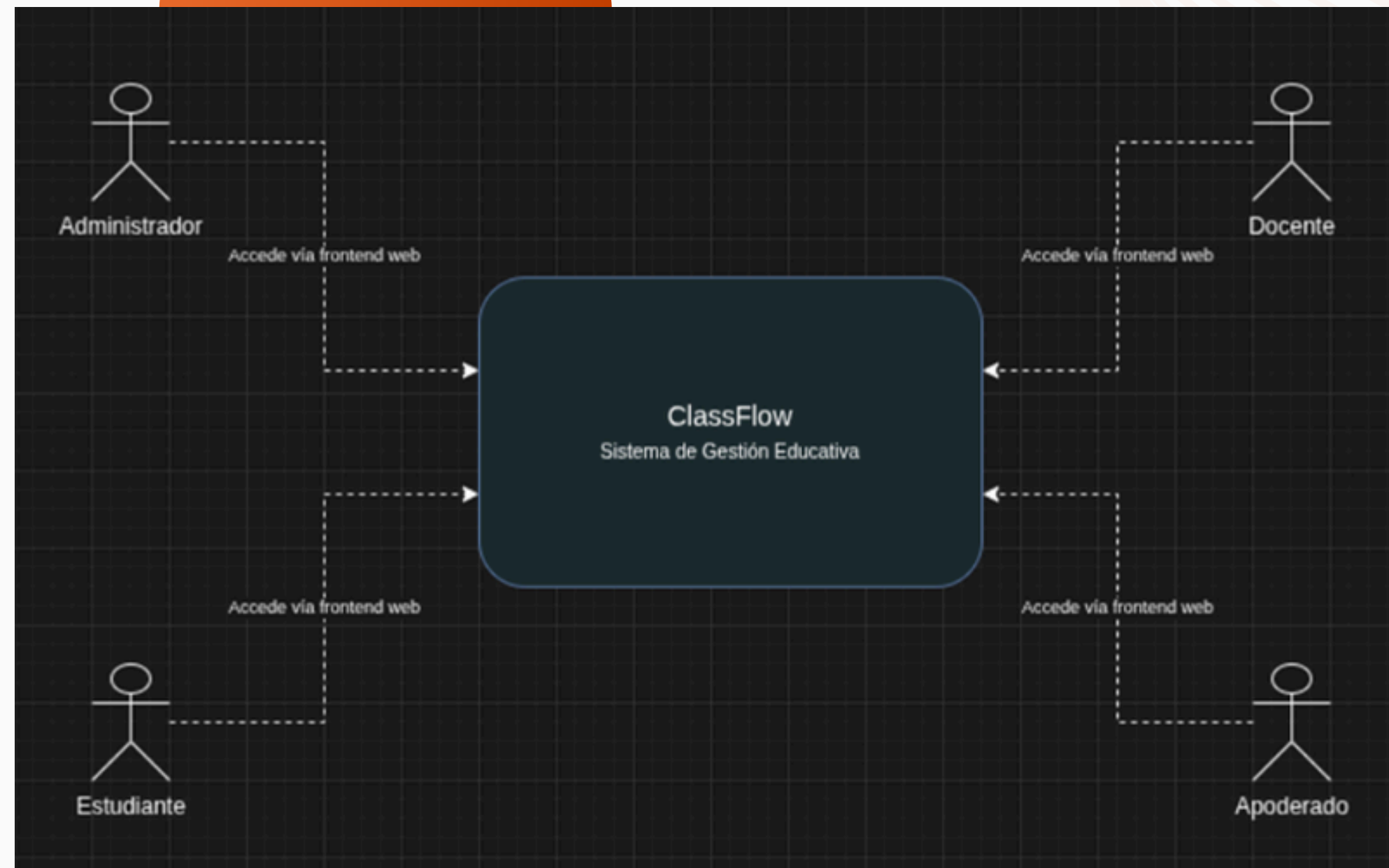
El sistema ClassFlow sigue un modelo con 4 actores principales: Administrador, Docente, Estudiante y Apoderado. Todos acceden mediante una SPA web que se comunica con el sistema, integrando microservicios, bases de datos, API Gateway y BFF.



4 actores principales interactúan con el sistema: Administrador gestiona usuarios y configuración; Docente registra asistencia y evaluaciones; Estudiante consulta calificaciones; Apoderado recibe notificaciones y mensajes.



El sistema integra API Gateway como punto único de entrada, BFF para agregación de datos en dashboards, microservicios especializados y bases de datos PostgreSQL independientes por servicio.



Arquitectura Técnica Contenedores y Comunicación

El Frontend SPA (React 18 + TypeScript) es servido por Nginx en el puerto 3000. El API Gateway (Spring Cloud Gateway) opera en el puerto 8080 como único punto de entrada. El BFF (Spring WebFlux) en el puerto 8086 agrega datos para dashboards.

El sistema cuenta con 5 microservicios independientes, cada uno con su propia base de datos PostgreSQL. La comunicación es REST síncrona a través del API Gateway; el frontend nunca accede directamente a los microservicios, garantizando seguridad y desacoplamiento.



Frontend → API Gateway (8080) → Microservicios

BFF (8086): Agregación de datos para vistas consolidadas

5 PostgreSQL independientes – Database per Service

Microservicios Principales

ms-academic gestiona cursos y evaluaciones.

ms-auth (Puerto 8081)

Servicio de autenticación y gestión de usuarios. Emite y valida tokens JWT con firma HS256. Administra roles: ADMIN, DOCENTE, ESTUDIANTE y APODERADO. Contraseñas con hashing BCrypt.

ms-academic gestiona cursos y evaluaciones.

ms-academic (Puerto 8082)

Administra cursos, asignaturas, evaluaciones y calificaciones. Centraliza el historial académico de estudiantes. Expone endpoints REST consumidos vía API Gateway. Base de datos PostgreSQL propia con migraciones Flyway.

ms-assistance registra asistencia

ms-assistance (Puerto 8083)

Registro de asistencia diaria por curso y docente. Gestión de anotaciones y observaciones académicas. Persistencia independiente en PostgreSQL. Integrado con BFF para reportes consolidados en dashboards.

ms-message gestiona mensajes

ms-message (8084): Mensajería interna entre usuarios del sistema. Gestiona el envío y recepción de anuncios y comunicaciones entre docentes, estudiantes y apoderados dentro de la plataforma ClassFlow.

ms-notification gestion de notificaciones

ms-notification (8085): Notificaciones push y email. Responsable de alertar a los usuarios sobre eventos académicos relevantes, calificaciones, asistencia y mensajes nuevos de forma automática.

bff/ API Gateway

BFF (8086) y API Gateway (8080): El Gateway actúa como punto único de entrada con enrutamiento y seguridad. El BFF agrega datos de múltiples microservicios para los dashboards del frontend.

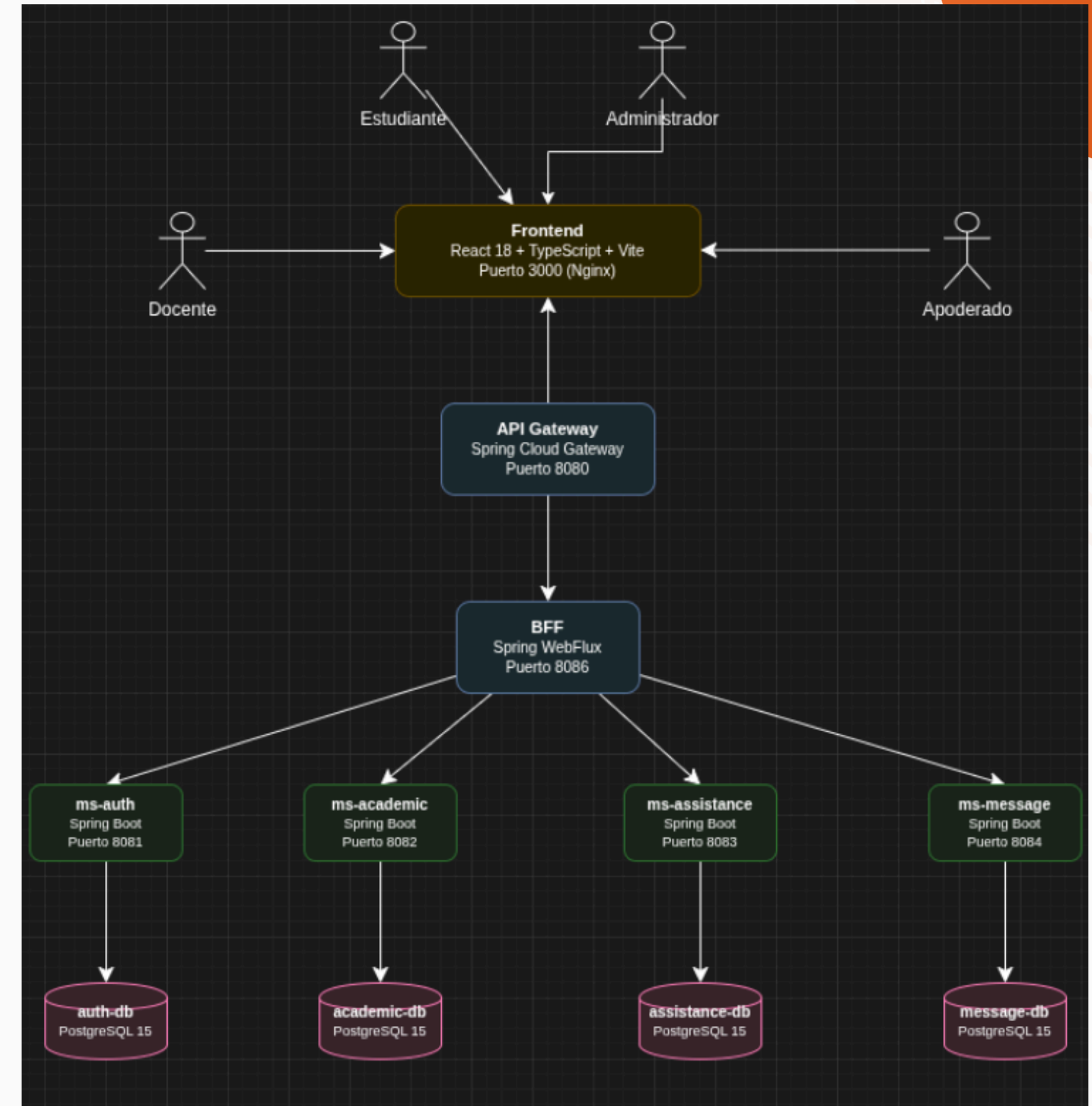
Patrón Database per Service

Aislamiento de Datos

5 bases de datos PostgreSQL independientes, cada una con volúmenes persistentes propios. Sin conexiones directas ni claves foráneas cruzadas entre microservicios. Migraciones Flyway versionadas para evolución controlada del esquema. Perfiles: local con H2 en memoria y docker con PostgreSQL.

Beneficios Clave

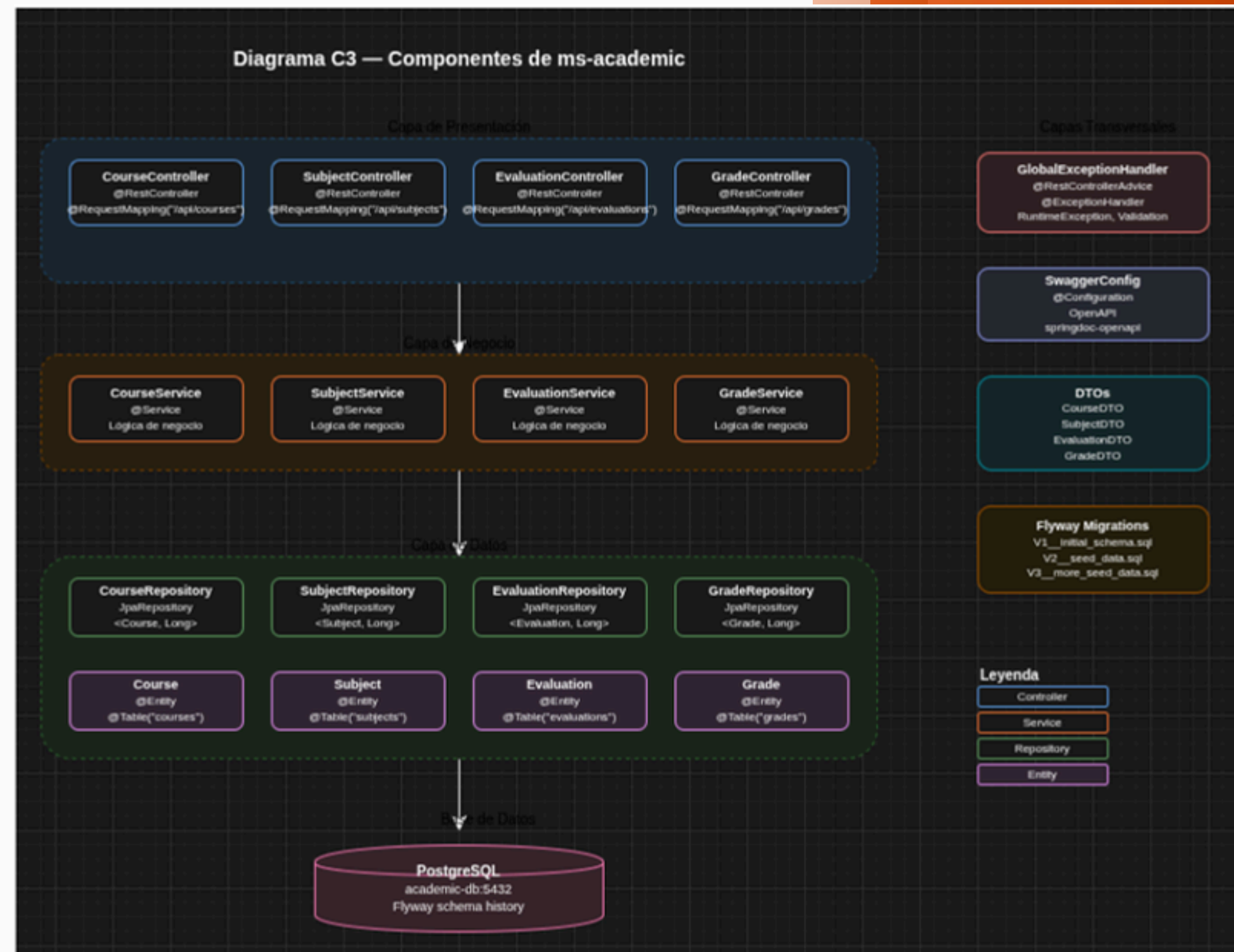
Escalabilidad independiente por servicio sin afectar otros módulos. Mayor seguridad al limitar el alcance de cada base de datos. Independencia de despliegue: cada microservicio puede actualizarse sin riesgo de ruptura de esquema compartido.



Cada microservicio (ms-auth, ms-academic, ms-assistance, ms-message, ms-notification) gestiona su propia base de datos. Este patrón garantiza bajo acoplamiento, alta cohesión y facilita la evolución independiente de cada componente del sistema ClassFlow.

Estructura Interna de un Microservicio

Cada microservicio sigue un patrón de capas común, ilustrado con el ejemplo de ms-academic. Esta arquitectura interna garantiza separación de responsabilidades, mantenibilidad y facilidad de prueba en todos los servicios del sistema.



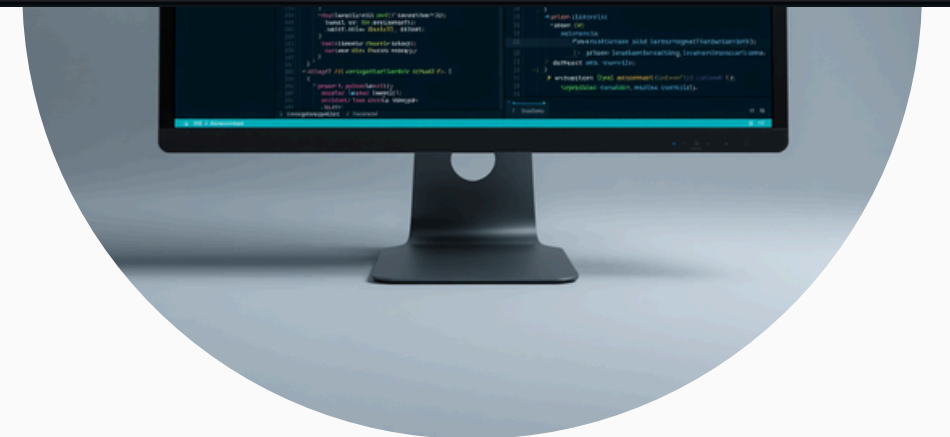
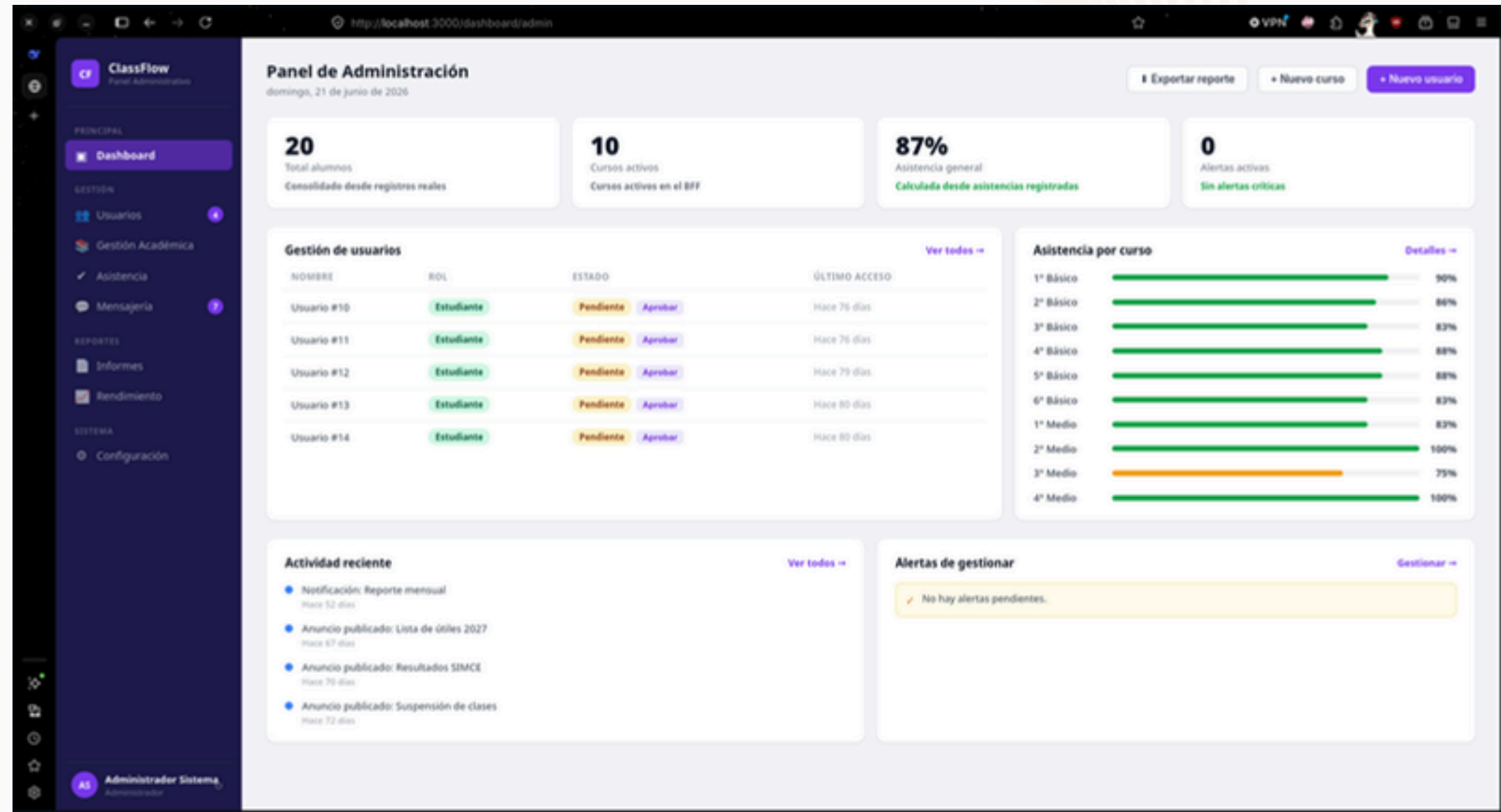
Frontend React 18 + TypeScript

Arquitectura SPA modular con páginas, router, services, context, hooks y components. Rutas protegidas por rol mediante ProtectedRoute (ADMIN, TEACHER, STUDENT, GUARDIAN). Autenticación JWT almacenado en localStorage y enviado en header Authorization.

Modularidad: pages, router, services, context, hooks

JWT en localStorage + header Authorization

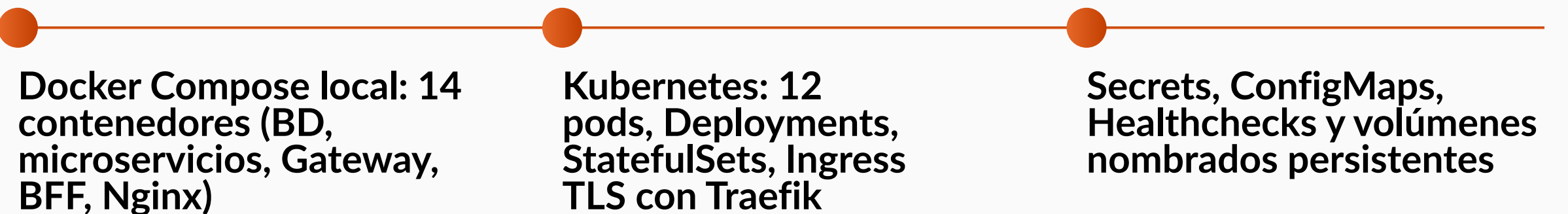
BFF para dashboards y CSS modular responsivo





Infraestructura y Despliegue

El sistema opera en dos ambientes: local con 14 contenedores Docker en red interna, y producción en Kubernetes con 12 pods en el namespace classflow. Se utilizan Secrets, ConfigMaps, Healthchecks y volúmenes persistentes para garantizar seguridad, configuración y disponibilidad.



Seguridad y Control de Acceso



Autenticación JWT con firma HS256 y validez de 24 horas. Roles diferenciados (ADMIN, DOCENTE, ESTUDIANTE, APODERADO) con control de acceso aplicado tanto en el frontend como en cada microservicio del backend.



Contraseñas protegidas con hashing BCrypt y salt aleatorio. CORS habilitado en el API Gateway para controlar orígenes permitidos. TLS activo en el entorno de producción mediante Traefik Ingress.



Prevención de inyección SQL mediante consultas parametrizadas con JPA/Hibernate. CSRF deshabilitado por diseño stateless de la API REST. Secrets de Kubernetes para gestión segura de credenciales.

Calidad, Pruebas y Documentación

Documentación Swagger/OpenAPI accesible en cada microservicio y en el API Gateway. Manejo global de errores con respuestas JSON estandarizadas. Seguimiento centralizado con GlitchTip y logging estructurado con SLF4J.

44 clases de prueba con JUnit 5 y Mockito, cubriendo controladores, servicios y repositorios. Estrategia de testing con perfiles locales y dockerizados para validación integral del sistema.

JUnit 5 + Mockito — Cobertura de controladores, servicios y repositorios

Swagger/OpenAPI — Documentación interactiva por microservicio

GlitchTip + SLF4J — Monitoreo de errores y logging centralizado



GlitchTip

classflow

Issues

Uptime Monitors

Performance

Logs

Projects

Releases

Organization

Profile

Support

GlitchTip 6.2.0

http://localhost:8000/classflow/issues/1

environment: docker server_name: b5b88fd5169f

Exception (most recent call first) Full Raw

RuntimeException

Error de prueba – Sentry/GlitchTip funciona correctamente!

mechanism: SpringExceptionHandlerResolver handled: no

com.ohiggins.classflow.auth.controller.AuthController in debugSentry at line 261

jdk.internal.reflect.DirectMethodHandleAccessor in invoke

java.lang.reflect.Method in invoke

org.springframework.web.method.support.InvocableHandlerMethod in doInvoke at line 258

org.springframework.web.method.support.InvocableHandlerMethod in invokeForRequest at line 191

org.springframework.web.servlet.mvc.method.annotation.ServletInvocableHandlerMethod in invokeAndHandle at line 118

org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter in invokeHandlerMethod at line 991

org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter in handleInternal at line 896

org.springframework.web.servlet.mvc.method.AbstractHandlerMethodAdapter in handle at line 87

org.springframework.web.servlet.mvc.method.AbstractHandlerMethodAdapter in doDispatch at line 1089

org.springframework.web.servlet.DispatcherServlet in doService at line 979

org.springframework.web.servlet.FrameworkServlet in processRequest at line 1014

org.springframework.web.servlet.FrameworkServlet in doGet at line 903

jakarta.servlet.http.HttpServlet in service at line 564

org.springframework.web.servlet.FrameworkServlet in service at line 885

1 minute ago

July 13, 2026, 12:35:11 PM GMT-4

Tags

server_name: b5b88fd5169f: 100%

environment: docker: 100%

</> Send

Response >> () JSON 400 Bad Request 10ms 1988 ...

1 {

2 "timestamp": "2026-07-13T16:23:11.341324362",

3 "status": 400,

4 "error": "Bad Request",

5 "message": "Error de prueba – Sentry/GlitchTip funciona correctamente!",

6 "path": "/api/auth/debug/sentry"

7 }

v4 Migration

Search

Cookies

Dev Tools

v3.5.2

Gracias por su atención

Presentado por:

Angelo Millan & Evens Reneus

